

РЕФЕРАТ

Расчетно-пояснительная записка 46 с., 16 рис., 2 табл., TODO источн., 2 прил.

ТРАССИРОВКА ПУТИ, ТУМАН, НЕРАВНОМЕРНЫЙ ТУМАН, ОТРАЖЕНИЯ, МОДЕЛЬ ОСВЕЩЕНИЯ, C++, ПУЛ ПОТОКОВ, РЕНДЕР ЧАСТЯМИ

Объектом исследования являются методы генерации реалистичного изображения сцены и методы визуализации неравномерного тумана в компьютерной графике

Цель работы — разработать программное обеспечение, использующее алгоритм трассировки пути, для генерации реалистичного изображения сцены, состоящей из геометрических объектов и неравномерного тумана.

В ходе работы были изучены и реализованы методы объёмного рендеринга, моделирования неоднородных рассеивающих сред, алгоритмы сэмплирования по экспоненциальному распределению для работы со средами с переменной плотностью. Реализовано программное обеспечение для реалистичной визуализации неоднородного тумана с использованием метода трассировки путей. Для моделирования пространственной изменчивости плотности среды применён процедурный шум Перлина, облачную структуру тумана.

СОДЕРЖАНИЕ

РЕФЕРАТ	4
ВВЕДЕНИЕ	7
1 Аналитическая часть	8
1.1 Анализ предметной области	8
1.1.1 Физические основы и математическое моделирование . .	8
1.1.2 Задачи визуализации и применение	9
1.2 Методы моделирования тумана	10
1.2.1 Атмосферное затухание	10
1.2.2 Объемный туман	10
1.2.3 Выбор метода генерации тумана	12
1.3 Модель освещения	12
1.3.1 Модель Фонга	12
1.3.2 Модель Блинна-Фонга	13
1.3.3 Глобальное освещение	14
1.4 Алгоритмы генерации кадра	14
1.4.1 Растеризация	14
1.4.2 Реймарчинг	15
1.4.3 Обратная трассировка лучей	15
1.4.4 Трассировка путей	16
1.4.5 Сравнительный анализ и выбор метода	16
1.5 Методы оптимизации рендера	17
1.5.1 Иерархия ограничивающих объёмов	17
1.5.2 Многопоточная обработка	17
2 Конструкторская часть	19
2.1 Требования к программному обеспечению	19
2.2 Модули программного обеспечения	19
2.3 Разработка алгоритмов	20
2.3.1 Алгоритм трассировки пути	20
2.3.2 Алгоритм генерации луча	21
2.3.3 Рекурсивная трассировка	22
2.4 Алгоритм визуализации тумана	24

3	Технологическая часть	28
3.1	Средства реализации	28
3.2	Реализация проведения исследования	28
3.3	Реализация алгоритмов	29
3.4	Пользовательский интерфейс	29
3.4.1	Панель настройки объектов	30
3.4.2	Панель изменения материала	30
3.4.3	Панель управления камерой	31
3.4.4	Панель настроек генерации	32
3.4.5	Панель добавления тумана	34
4	Исследовательская часть	36
4.1	Цель и методика исследования	36
4.2	Технические характеристики	36
4.3	Результаты исследований	37
4.3.1	Зависимость времени рендеринга от количества лучей	37
4.3.2	Зависимость времени генерации кадра от количества пе- реотражений	37
4.4	Вывод	38
	ЗАКЛЮЧЕНИЕ	39
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	42
	ПРИЛОЖЕНИЕ А	43
	ПРИЛОЖЕНИЕ Б	44

ВВЕДЕНИЕ

Компьютерная графика является важной составляющей в различных сферах человеческой деятельности, например: киноиндустрия, разработка видеоигр, архитектурная визуализация, а также профессиональные симуляторы.

Туман представляет собой сложный оптический эффект, возникающий в результате взаимодействия света с взвешенными частицами воды в атмосфере. На корректную визуализацию влияют 3 фактора среды: поглощение света, рассеяние луча и затухание света [1]. Современные методы визуализации позволяют добиться высокого уровня фотореализма, однако требуют значительных вычислительных ресурсов.

Целью данной работы является разработка программного обеспечения для визуализации сцены из геометрических объектов и неравномерного тумана.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать существующие методы моделирования тумана и алгоритмы рендеринга. Провести сравнительный анализ и выбрать подходы для реализации;
- разработать алгоритм генерации изображения с поддержкой взаимодействия лучей с туманом;
- реализовать алгоритмы генерации изображения;
- реализовать модель объемного тумана;
- провести исследование влияния параметров алгоритма на производительность.

1 Аналитическая часть

1.1 Анализ предметной области

1.1.1 Физические основы и математическое моделирование

Туман представляет собой атмосферное явление, возникающее из-за скопления мелких капель воды в атмосфере. Взаимодействие света с частицами тумана описывается двумя основными типами рассеяния: рассеянием Рэлея и рассеянием Ми. Рассеяние Рэлея происходит, когда размер рассеивающих частиц значительно меньше длины волны падающего света. Интенсивность такого рассеяния пропорциональна λ^{-4} , где λ — длина волны света [2]. Рассеяние Ми используется в случаях, когда размер частиц сопоставим с длиной волны или превышает её, что характерно для капель воды в тумане.

Для описания поведения света в среде используется уравнение переноса излучения [3], представленное в формуле 1

$$w\nabla L(x, \omega) = -\sigma_t(x)L(x, \omega) + \sigma_s(x)L_s(x, \omega) + \sigma_a(x)L_e(x, \omega), \quad (1)$$

где $L(x, \omega)$ — яркость в точке x по направлению ω , $\sigma_t = \sigma_a + \sigma_s$ — коэффициент затухания (сумма коэффициентов поглощения σ_a и рассеяния σ_s), $L_s(x, \omega)$ — яркость, полученная за счёт рассеяния внутри среды, $L_e(x, \omega)$ — собственное излучение среды.

Первый член правой части описывает затухание света, второй — рассеяние, третий — излучение самой среды.

Затухание света при прохождении через однородную среду описывается законом Бера-Ламберта [4]:

$$I(d) = I_0 \exp(-\sigma_t d), \quad (2)$$

где I_0 — начальная интенсивность света, $I(d)$ — интенсивность после прохождения расстояния d через среду, σ_t — коэффициент затухания.

Направленность рассеяния описывается фазовой функцией $p(\omega_i, \omega_o)$,

определяющей вероятность того, что свет, падающий по направлению ω_i , будет рассеян в направлении ω_o . Для изотропного рассеяния $p = 1/(4\pi)$. Для анизотропного рассеяния часто используется фазовая функция Хенни-Гринштейна [5], приведена в формуле 3:

$$p_{\text{HG}}(\cos \theta, g) = \frac{1}{4\pi} \frac{1 - g^2}{(1 + g^2 - 2g \cos \theta)^{3/2}}, \quad (3)$$

Где θ — угол рассеяния, $g \in [-1, 1]$ — параметр анизотропии ($g > 0$ — преимущественно прямое рассеяние, $g < 0$ — обратное, $g = 0$ — изотропное).

1.1.2 Задачи визуализации и применение

Визуализация тумана требует решения множества взаимосвязанных задач. Физическая корректность достигается путём численного решения уравнения переноса излучения методом Монте-Карло [6], что требует трассировки множества лучей и вычисления интегралов по всем направлениям [3]. Вычислительная эффективность является важным фактором, так как объемный рендеринг требует вычисления множества точек взаимодействия для каждого луча. Использование структур ускорения, таких как иерархия ограничивающих объемов (BVH), позволяет сократить время поиска пересечений.

Моделирование неравномерности тумана достигается [7] за счет применения процедурных методов генерации плотности на основе шума Перлина [8], что создаёт реалистичные области с различной концентрацией частиц. Для неравномерного тумана плотность описывается функцией $\rho(x, y, z)$, и коэффициенты затухания становятся пространственно-зависимыми.

В практических приложениях используются упрощённые модели: линейный туман, где затухание линейно зависит от расстояния, экспоненциальный туман, соответствующий закону Бера-Ламберта, и экспоненциальный квадратный туман, обеспечивающий более плавное затухание [7]. Однако для достижения фотореализма необходима реализация объемного рендеринга с решением уравнения переноса излучения.

1.2 Методы моделирования тумана

1.2.1 Атмосферное затухание

При использовании атмосферного затухания плотность частиц во всем объеме тумана постоянна. Коэффициент тумана f определяет степень видимости объекта, где $f = 1$ соответствует отсутствию тумана, а $f = 0$ — полному поглощению света. Итоговый цвет [7] вычисляется по формуле 4.

$$C_{\text{final}} = f \cdot C_{\text{obj}} + (1 - f) \cdot C_{\text{fog}}, \quad (4)$$

где C_{obj} — цвет объекта, C_{fog} — цвет тумана.

Линейный туман. Интенсивность линейно зависит от расстояния [9]:

$$f = \frac{d_{\text{end}} - d}{d_{\text{end}} - d_{\text{start}}}, \quad (5)$$

где d — расстояние до точки, d_{start} и d_{end} — границы действия тумана. Модель проста в реализации, но не соответствует физическим законам.

Экспоненциальный туман. Основан на законе Бера-Ламберта и является упрощенной физической моделью затухания [7]:

$$f = \exp(-\rho \cdot d), \quad (6)$$

где $\rho \in [0, 1]$ — плотность тумана.

Экспоненциальный квадратный туман. Обеспечивает более резкое затухание на больших расстояниях [7]:

$$f = \exp(-\rho^2 \cdot d^2). \quad (7)$$

1.2.2 Объемный туман

Объёмный туман представляет собой метод, при котором туман моделируется как трёхмерная среда, заполняющая определённый объём пространства. При трассировке луча через объём расстояние до точки взаимодействия определяется стохастически [10].

$$t = -\frac{\ln(\xi)}{\sigma_t}, \quad (8)$$

где $\xi \sim U(0, 1)$ — случайная величина, равномерно распределённая на $[0, 1]$, а σ_t — коэффициент затухания среды.

В точке взаимодействия [11] луч либо поглощается, либо рассеивается в новом направлении ω' , выбранном согласно фазовой функции $p(\omega, \omega')$. Точка взаимодействия определяется по формуле (9).

$$p = t_o + hit_dist/ray_len \quad (9)$$

где

- t_o — это точка вхождения луча в объем;
- hit_dist — расстояние до точки пересечения с туманом;
- ray_len — длина луча.

Для неоднородных сред с переменной плотностью $\rho(p)$ коэффициент затухания становится пространственно-зависимым:

$$\sigma_t(p) = \sigma_{t,0} \cdot \rho(p) \quad (10)$$

Для моделирования реалистичных неоднородностей применяется шум Перлина — градиентная функция, генерирующая псевдослучайные значения с плавными переходами [8].

$$\text{turbulence}(p, n) = \sum_{i=0}^{n-1} \frac{|\text{noise}(2^i \cdot p)|}{2^i}, \quad (11)$$

где:

- $p = (x, y, z)$ — точка в пространстве;
- noise — функция генерации шума;
- n — количество октав.

Плотность неравномерного тумана определяется по формуле (12)

$$\rho(p) = \rho_0 \cdot \text{turbulence}(s \cdot p, n), \quad (12)$$

где ρ_0 — базовая плотность, s — масштабный коэффициент.

Для моделирования приземного тумана добавляется зависимость от высоты, формула (13):

$$\rho(p) = \rho_0 \cdot \text{turbulence}(s \cdot p, n) \cdot \exp(-k \cdot \max(y, 0)), \quad (13)$$

где $k > 0$ — коэффициент спада по высоте.

1.2.3 Выбор метода генерации тумана

В таблице 1 приведено сравнение методов моделирования тумана.

Таблица 1 – Сравнение методов моделирования тумана

Критерий	Линейный	Экспонен.	Экспонен. кв.	Объемный
Физическая корректность	Низкая	Высокая	Средняя	Высокая
Вычисл. сложность	Очень низкая	Низкая	Низкая	Высокая
Реалистичность	Низкая	Средняя	Средняя	Высокая
Неоднородность	Нет	Нет	Нет	Да

Для дальнейшей работы была выбрана модель объемного тумана на основе шума Перлина.

1.3 Модель освещения

1.3.1 Модель Фонга

Модель Фонга [12, с. 33] — классическая модель локального освещения, предложенная в 1975 году. В общем виде модель освещения Фонга состоит из суммы фоновой, диффузной и зеркальной составляющих и имеет следующий вид:

$$I = I_a + I_d + I_s = m_a \cdot L_a + m_d \cdot k_d \cdot L_d + m_s \cdot k_s \cdot L_s, \quad (14)$$

где I_a , I_d , I_s — интенсивности фоновой, диффузной и зеркальной составляющих, m_a , m_d , m_s — коэффициенты материала, L_a , L_d , L_s — интенсивности

источников света, k_d — коэффициент диффузного отражения, k_s — коэффициент зеркального отражения.

На рисунке 1 представлено сравнение компонент освещения на примере сферы: фоновая компонента закрашивает сферу одним цветом, диффузная закрашивает градиентом цвета, а зеркальная только добавляет пятно блика.

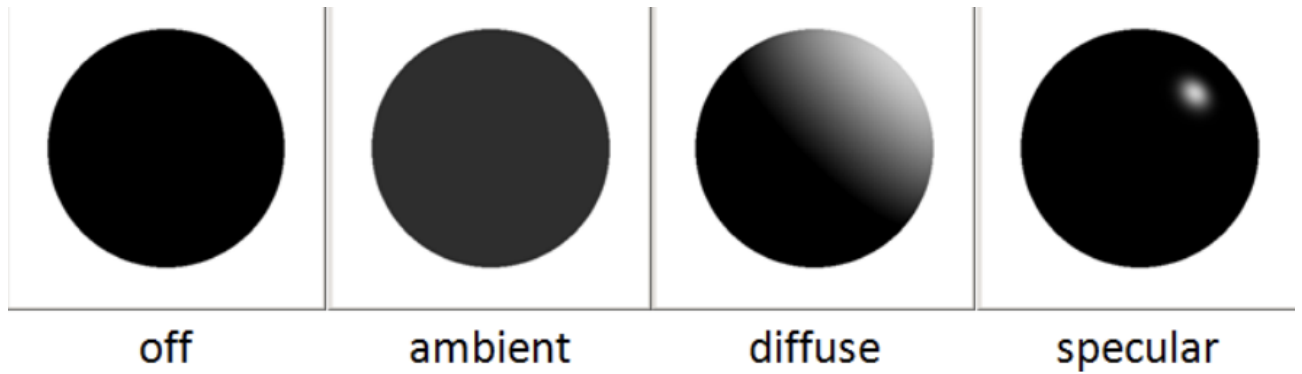


Рисунок 1 — Фоновая, диффузная и зеркальная компоненты.

1.3.2 Модель Блинна-Фонга

Модель Блинна-Фонга — модификация модели Фонга, предложена для повышения производительности [13]. Вместо вектора отражения используется вектор-полусумма H между направлениями на источник света и наблюдателя:

$$I = I_a k_a + I_l k_d \max(0, n \cdot L) + I_l k_s \max(0, n \cdot H)^n, \quad (15)$$

где вектор-полусумма определяется как:

$$H = \frac{L + V}{|L + V|}, \quad (16)$$

где:

- n — вектор нормали к поверхности;
- V — вектор от поверхности к наблюдателю.

Преимущества модели Блинна — Фонга:

- уменьшении времени выполнения реализации алгоритма;
- для фиксированных источника света H может быть предвычислен.

1.3.3 Глобальное освещение

Глобальное освещение — общий термин для алгоритмов рендеринга, учитывающих полное световое взаимодействие в сцене, включая прямой и непрямой свет. В отличие от локальных моделей освещения, которые вычисляют только прямое освещение от источников света, глобальное освещение моделирует многократные переотражения света между объектами сцены. Математически [14] глобальное освещение описывается формулой (17)

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} f_r(x, \omega_i, \omega_o) L_i(x, \omega_i) (n \cdot \omega_i) d\omega_i, \quad (17)$$

где:

- $L_o(x, \omega_o)$ — исходящая яркость в направлении ω_o ;
- $L_e(x, \omega_o)$ — собственное излучение поверхности;
- $f_r(x, \omega_i, \omega_o)$ — BRDF — функция, определяющая визуальные свойства материала;
- $L_i(x, \omega_i)$ — входящая яркость из направления ω_i ;
- n — нормаль к поверхности;
- Ω — полусфера над точкой x .

1.4 Алгоритмы генерации кадра

1.4.1 Растеризация

Растеризация — это процесс, с помощью которого примитив преобразуется в двумерное изображение [15]. Для каждого пикселя вычисляется цвет с использованием локальных моделей освещения.

Преимущества: высокая производительность, предсказуемое время визуализации.

Недостатки: не поддерживает физически корректные отражения, преломления, мягкие тени и глобальное освещение.

1.4.2 Реймарчинг

Реймарчинг — метод для визуализации объектов, заданных функциями знакового расстояния (SDF). Луч итеративно продвигается вдоль направления на расстояние, равное минимальному расстоянию до поверхности:

$$t_{i+1} = t_i + d_{\text{scene}}(p_i), \quad p_i = o + t_i d, \quad (18)$$

где o — начало луча, d — направление, $d_{\text{scene}}(p)$ — функция знакового расстояния до ближайшей поверхности сцены, t_i — пройденное расстояние вдоль луча к шагу i .

Итерации продолжаются до тех пор, пока расстояние не станет меньше заданного порога ε или не будет превышено максимальное расстояние [16].

Применение: фракталы, процедурная геометрия, объёмные эффекты, конструктивная твёрдотельная геометрия.

Недостатки: медленнее традиционной трассировки для полигональных моделей.

1.4.3 Обратная трассировка лучей

В алгоритме обратной трассировки лучей [17] лучи испускаются из камеры через каждый пиксель и ищут пересечения с объектами сцены. При попадании на поверхность луч может порождать вторичные лучи: отражённые, преломлённые и теневые. Отражённый луч продолжает трассировку в направлении зеркального отражения, преломлённый — проходит сквозь прозрачные объекты согласно закону Снеллиуса, теневой — проверяет видимость источника света для вычисления освещённости.

Цвет пикселя вычисляется рекурсивно с учётом локального освещения и вклада от вторичных лучей до достижения максимальной глубины рекурсии или фонового цвета.

Преимущества: точные зеркальные отражения, преломления, резкие тени.

Недостатки: не моделирует диффузное переотражение света, каустику и мягкие тени без модификаций; вычислительно затратен.

1.4.4 Трассировка путей

Трассировка путей — метод решения уравнения рендеринга методом Монте-Карло [6], обеспечивающий физически корректное глобальное освещение [18]. Уравнение рендеринга в точке x для направления ω_o имеет вид, указанный на формуле (17)

Для численного решения интеграла методом Монте-Карло для каждого пикселя испускается множество случайных лучей. Каждый луч отражается от поверхностей, изменяя направление согласно методу выборки по значимости [19]. Луч продолжает случайные отражения до попадания на источник света, поглощения или превышения максимальной глубины.

Визуализация алгоритма трассировки путей представлена на рисунке 2

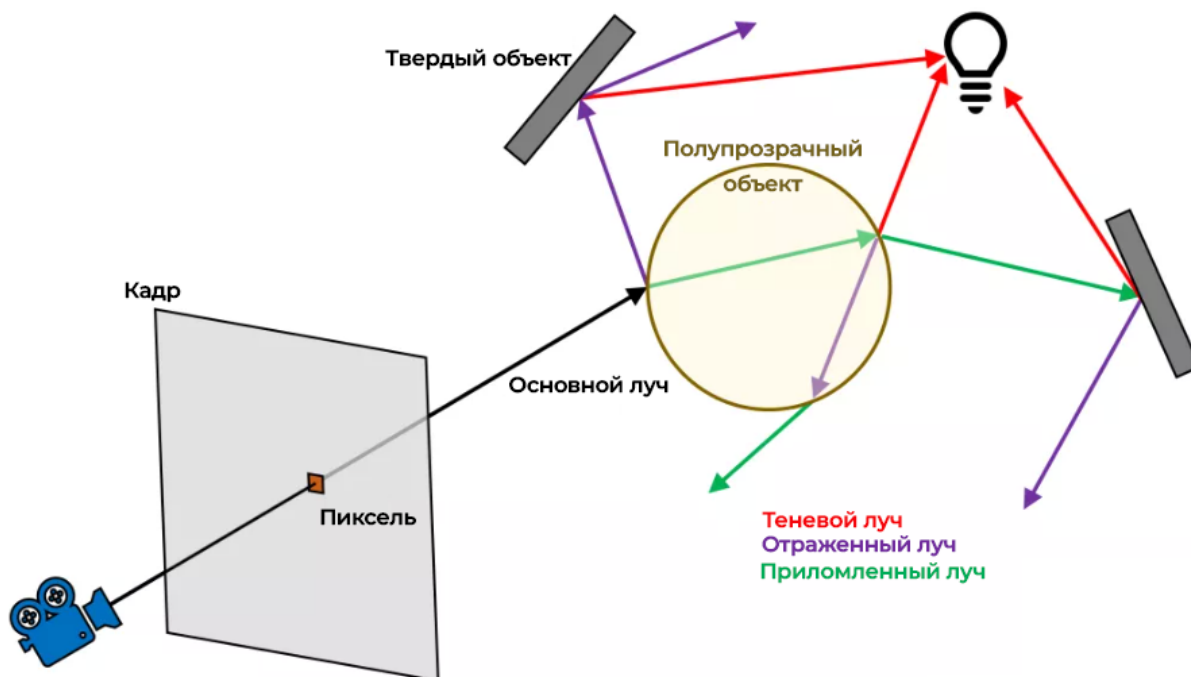


Рисунок 2 – Визуализация алгоритма трассировки путей

Преимущества: физически корректное глобальное освещение, мягкие тени, каустика, цветовое переотражение.

Недостатки: высокая трудоёмкость, требуется большое количество лучей на пиксель для снижения шума.

1.4.5 Сравнительный анализ и выбор метода

В таблице 2 приведено сравнение методов генерации кадра.

Таблица 2 – Сравнение методов рендеринга

Метод	Физ. корр.	Сложность	Глоб. осв.	Объёмы
Растеризация	Низкая	Очень низкая	Нет	Нет
Реймарчинг	Средняя	Высокая	Частично	Да
Трассировка лучей	Высокая	Средняя	Частично	Нет
Трассировка путей	Очень высокая	Очень высокая	Да	Да

Для курсовой работы выбрана трассировка путей, так как она обеспечивает физически корректное моделирование взаимодействия света с туманом, поддерживает объёмный рендеринг участвующих сред и позволяет достичь фотореалистичного качества изображения.

1.5 Методы оптимизации рендера

Трассировка путей является вычислительно затратным методом, требующим применения оптимизаций для достижения приемлемого времени рендеринга. Основные направления оптимизации включают ускорение поиска пересечений лучей с объектами, параллелизацию вычислений и улучшение отзывчивости пользовательского интерфейса.

1.5.1 Иерархия ограничивающих объёмов

Иерархия ограничивающих объёмов (BVH) — структура данных, организующая объекты сцены в виде дерева, где каждый узел содержит ограничивающий прямоугольник, охватывающий все объекты в поддереве. При трассировке луча сначала проверяется пересечение с ограничивающим объёмом узла; если пересечения нет, всё поддерево отбрасывается, что сокращает количество проверок [7].

1.5.2 Многопоточная обработка

Алгоритм трассировки путей можно выполнить параллельно в нескольких рабочих потоках, поскольку расчёт цвета каждого пикселя независим от других. Для эффективного использования многоядерных процессоров применяется пул потоков — набор рабочих потоков, обрабатывающих задачи из общей очереди.

Целесообразно разделить генерируемое изображение на части, и каждую из них рендерить параллельно. Получившиеся куски изображения независимо выводятся в интерфейс, что обеспечивает улучшение пользовательского опыта.

Вывод

В данной части проведён анализ теоретических основ визуализации тумана, моделей освещения, алгоритмов генерации изображения и методов оптимизации. Выполнен сравнительный анализ методов генерации изображения и визуализации тумана.

2 Конструкторская часть

2.1 Требования к программному обеспечению

Программное обеспечение должно обеспечивать следующие функциональные возможности:

- 1) добавление геометрического объекта из списка: сфера, цилиндр, параллелепипед, трехугольная и четырехугольная пирамида, конус, цилиндр;
- 2) добавление нескольких источников света;
- 3) добавление нескольких камер, с возможностью переключения между ними;
- 4) изменения материала объекта.

2.2 Модули программного обеспечения

Структура разрабатываем программы описана на рисунке 3.

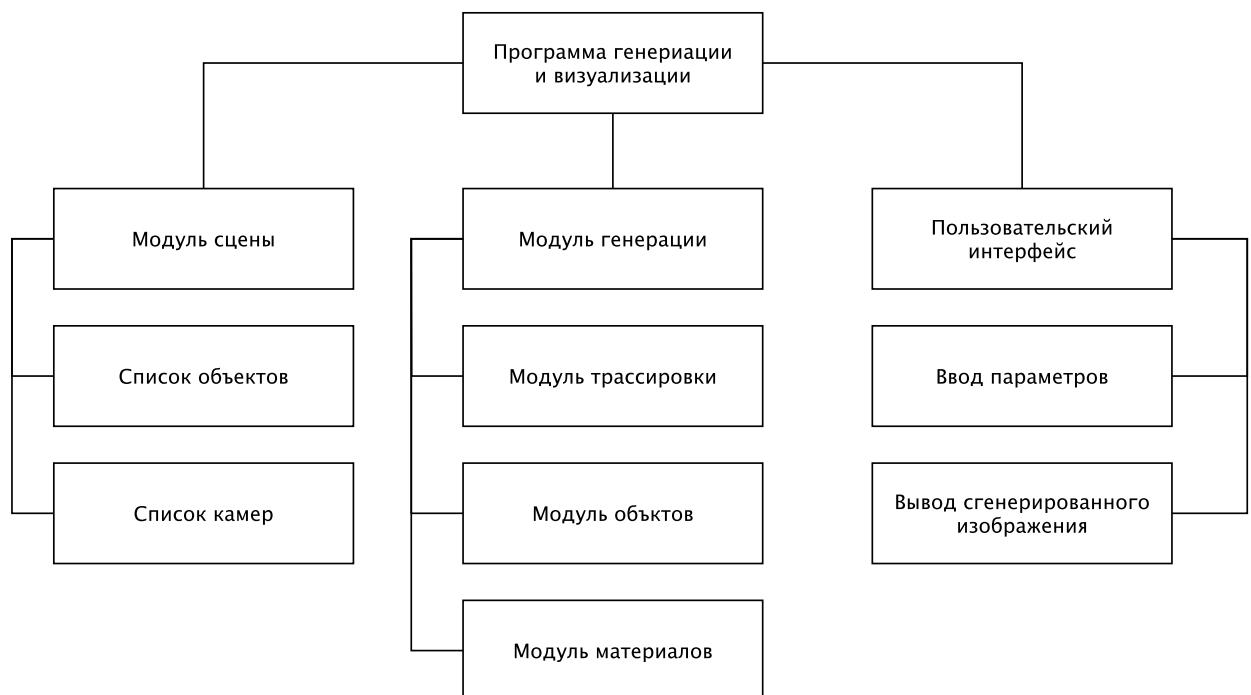


Рисунок 3 – Структурная схема программы

Разрабатываемую программу можно поделить на следующие модули:

- 1) интерфейс — графическая оболочка для взаимодействия с пользователем.
- 2) Модуль сцены — предоставляет методы для редактирования объектов сцены.
- 3) Модуль генерации изображения — генерирует итоговое изображение.

2.3 Разработка алгоритмов

2.3.1 Алгоритм трассировки пути

Процесс генерации изображения выполняется для заданной области, что обеспечивает возможность параллельной обработки. Для каждого пикселя в области испускается заданное количество лучей, и результаты усредняются для снижения шума методом Монте-Карло. Схема реализации алгоритма генерации части изображения представлена на рисунке 4

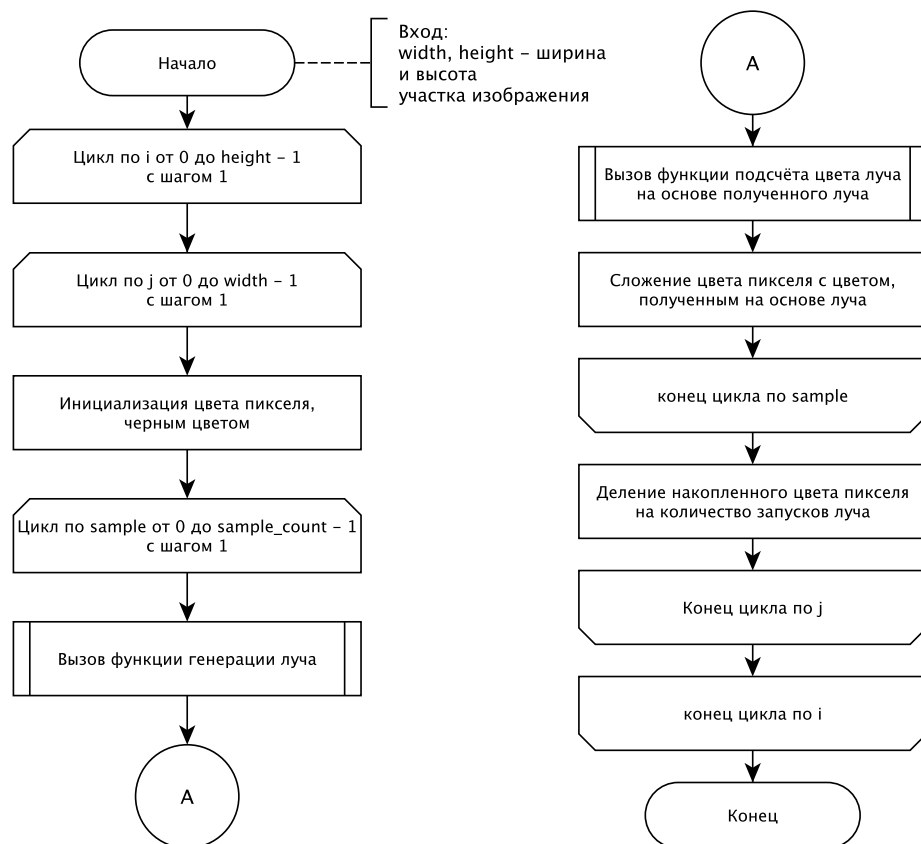


Рисунок 4 – Схема реализации алгоритма генерации части изображения

Цвет пикселя с координатами (i, j) вычисляется как среднее значение цвета по всем лучам:

$$C_{i,j} = \frac{1}{N_s} \sum_{k=1}^{N_s} L(r_k), \quad (19)$$

где N_s — количество лучей на пиксель, $L(r_k)$ — яркость, возвращаемая функцией трассировки для луча r_k .

2.3.2 Алгоритм генерации луча

Для каждого пикселя генерируется луч, проходящий через случайную точку внутри него, что реализует сглаживание. Координаты точки на плоскости изображения вычисляются как [11]:

$$p_{\text{sample}} = p_{00} + (i + \xi_x)\Delta u + (j + \xi_y)\Delta v, \quad (20)$$

где p_{00} — позиция центра пикселя $(0, 0)$, Δu и Δv — векторы, определяющие размер пикселя в горизонтальном и вертикальном направлениях, $\xi_x, \xi_y \sim U(0, 1)$ — случайные смещения для выборки Монте-Карло.

Начало луча определяется с учётом эффекта глубины резкости. Для камер, где угол размытия больше нуля начало луча случайным образом выбирается на диске размытия.

Направление луча вычисляется как:

$$d = p_{\text{sample}} - ray_origin, \quad (21)$$

где ray_origin это точка начала луча.

Схема алгоритма генерации луча представлена на рисунке 5

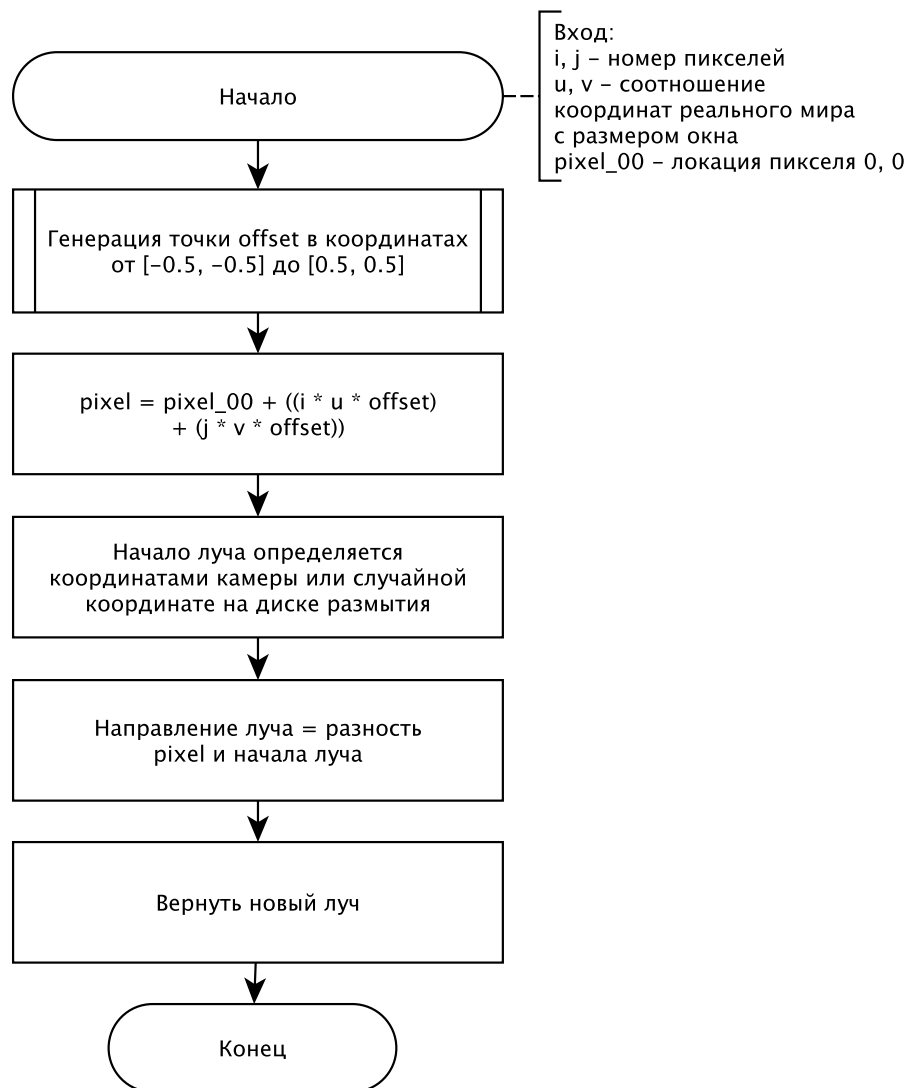


Рисунок 5 – Схема алгоритма генерации луча

2.3.3 Рекурсивная трассировка

Вычисление цвета луча выполняется рекурсивно с ограничением глубины рекурсии для предотвращения бесконечных отражений. Алгоритм реализует следующую логику:

- 1) проверка глубина рекурсии. Если глубина рекурсии превышена, возвращается черный цвет;
- 2) проверка пересечения луча со всеми объектами сцены. Если пересечений нет, возвращается черный цвет;
- 3) вычисления излучения материала в точке пересечения. Формула (22);

- 4) рассеивание луча. Материал определет, происходит ли рассеивание луча, вычисляет новое направление, и коэффициент затухания;
- 5) рекурсивный вызов функции. Для рассеянного луча рекурсивно вычисляется цвет с уменьшенной глубиной. Формула (23).

$$C_{\text{emit}} = L_e(p), \quad (22)$$

где p — координаты точки пересечения.

$$C_{\text{scatter}} = a \odot L(r', d - 1), \quad (23)$$

где $\odot$ — покомпонентное произведение векторов (цветов), r' — рассеянный луч с началом в точке пересечения, d — количество переотражений луча.

Финальный цвет луча определяется как сумма излучения и рассеянного света:

$$L(r, d) = C_{\text{emit}} + C_{\text{scatter}}. \quad (24)$$

Эта формула является дискретной аппроксимацией уравнения рендеринга, решаемого методом Монте-Карло.

Схема алгоритма рекурсивной трассировки пути представлена на рисунке 6

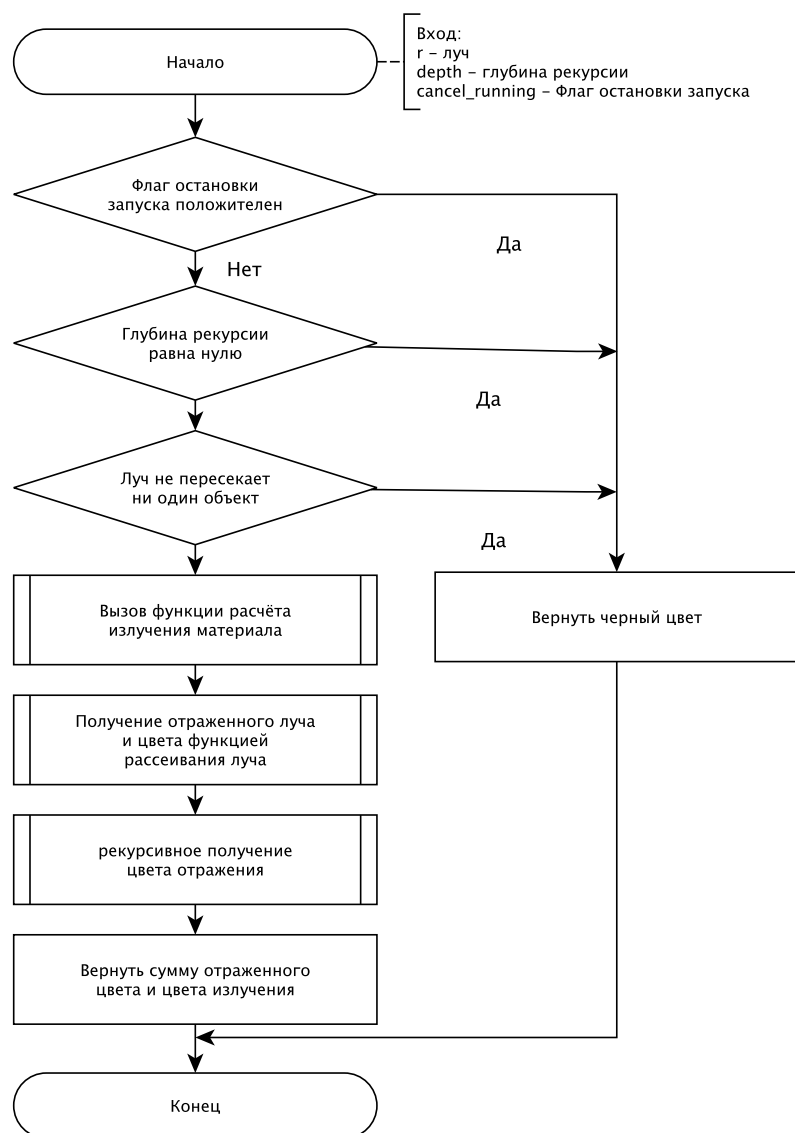


Рисунок 6 – Схема алгоритма рекурсивной трассировки пути

Рекурсивная структура алгоритма естественным образом моделирует глобальное освещение: луч может многократно отражаться от поверхностей, собирая непрямой свет, что создаёт реалистичные эффекты цветового переотражения и мягких теней.

2.4 Алгоритм визуализации тумана

Взаимодействие луча с объёмным туманом моделируется стохастически с учётом переменной плотности среды, что обеспечивает реалистичное поведение неоднородного тумана [20].

Проверка пересечения с границами. Сначала определяется интервал $[t_0, t_1]$, на котором луч находится внутри ограничивающего объёма тумана.

на:

$$t_0 = \max(t_{\text{entry}}, t_{\min}), \quad t_1 = \min(t_{\text{exit}}, t_{\max}), \quad (25)$$

где t_{entry} и t_{exit} — параметры входа и выхода луча из ограничивающего объёма, $[t_{\min}, t_{\max}]$ — допустимый интервал трассировки. Если $t_0 \geq t_1$, луч не проходит через объём тумана.

Вычисление расстояния до взаимодействия. Расстояние, на котором происходит взаимодействие фотона с частицами тумана, определяется методом инверсной функции распределения для экспоненциального закона [21]:

$$d_{\text{hit}} = -\frac{\ln(\xi)}{\rho_0}, \quad (26)$$

где $\xi \sim U(0, 1)$ — случайная величина, ρ_0 — базовая плотность тумана. Параметр луча, соответствующий точке взаимодействия:

$$t = t_0 + \frac{d_{\text{hit}}}{|d|}, \quad (27)$$

где $|d|$ — длина вектора направления луча. Если $d_{\text{hit}} > (t_1 - t_0) \cdot |d|$, взаимодействие не происходит в пределах объёма.

Вычисление локальной плотности. В точке взаимодействия $p = o + td$ вычисляется фактическая плотность с учётом неоднородности:

$$\rho(p) = \rho_0 \cdot f_{\text{noise}}(p) \cdot f_{\text{height}}(p), \quad (28)$$

где $f_{\text{noise}}(p)$ — функция шума Перлина, создающая турбулентность:

$$f_{\text{noise}}(p) = 0.2 + 0.8 \cdot |\text{noise}(s \cdot p)|, \quad (29)$$

s — масштабный коэффициент, а $f_{\text{height}}(p)$ — экспоненциальный множитель спада по высоте для приземного тумана:

$$f_{\text{height}}(p) = \exp(-k \cdot \max(y, -10)), \quad (30)$$

где $k > 0$ — коэффициент спада по высоте, $y = p_y$ — вертикальная координата.

Стохастическое отсеивание. Поскольку базовое расстояние d_{hit} вычислено для постоянной плотности ρ_0 , а фактическая плотность $\rho(p)$ может быть меньше, применяется стохастическое отсеивание [21]:

$$P_{\text{accept}} = \min \left(\frac{\rho(p)}{\rho_0}, 1 \right) = \min(\rho(p), 1). \quad (31)$$

Генерируется случайная величина $\xi' \sim U(0, 1)$; если $\xi' > P_{\text{accept}}$ или $\rho(p) < 10^{-5}$, взаимодействие отвергается и поиск продолжается.

Схема алгоритма генерации тумана представлена на рисунке 7

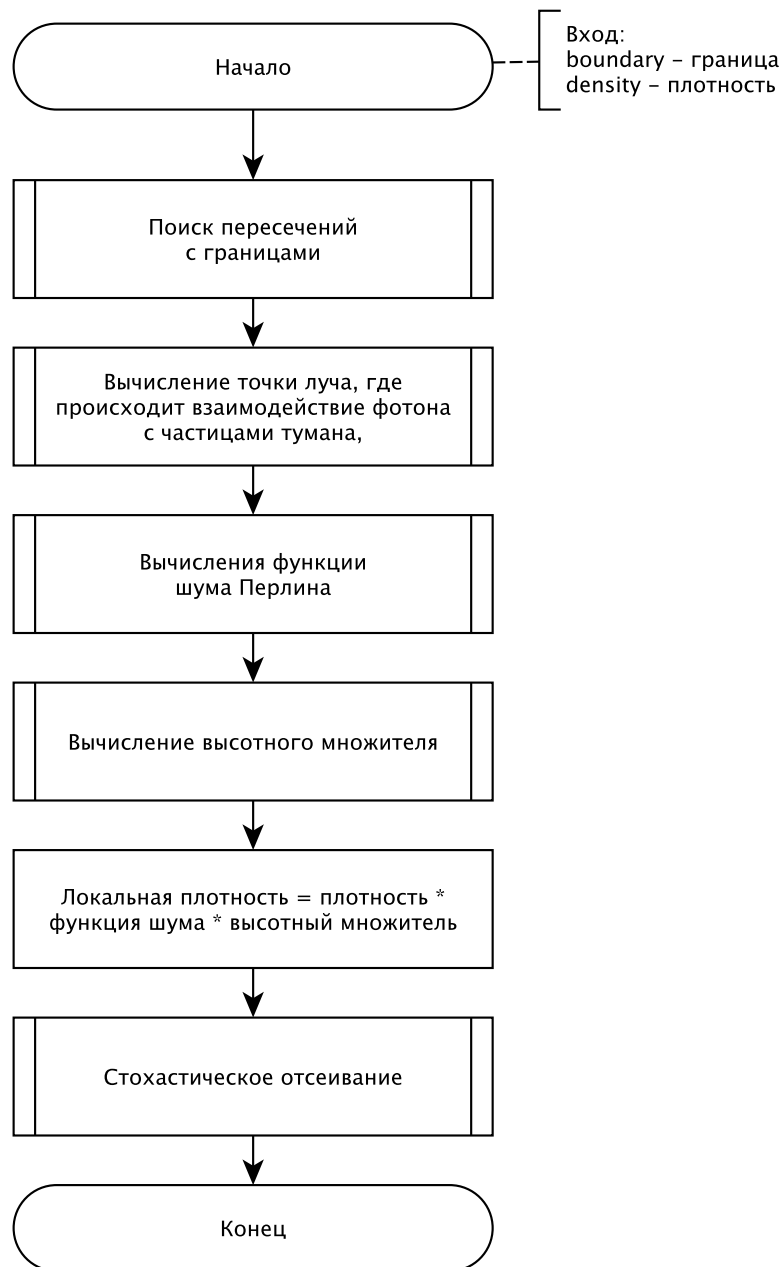


Рисунок 7 – Схема алгоритма генерации тумана

Вывод

В данной части были разработаны алгоритмы системы визуализации тумана методом трассировки путей. Описаны алгоритмы генерации изображения, формирования лучей и рекурсивной трассировки, спроектирована программа.

3 Технологическая часть

3.1 Средства реализации

Для реализации программного обеспечения выбран язык программирования C++20 [22]. Выбор языка обусловлен:

- наличием опыта работы с языком;
- наличием стандартной библиотеки (STL), которая предоставляет контейнеры для работы с данными;
- возможностью использования нативных потоков;
- поддержкой объектно-ориентированной парадигмы;
- кроссплатформенностью.

Для реализации многопоточной обработки применяются средства стандартной библиотеки C++ [22]: `std::thread` для создания потоков, `std::mutex` и `std::condition_variable` для синхронизации, `std::future` и `std::packaged_task` для асинхронного программирования, контейнеры `std::vector`, `std::array` и умные указатели `std::shared_ptr` для управления памятью.

Графический интерфейс реализован с использованием фреймворка Qt 6 [23]. Применяются модули: `QtWidgets` для создания оконного интерфейса, `QtConcurrent` для параллельного выполнения задач, `QFutureWatcher` для отслеживания асинхронных операций, `QPixmap` и `QPainter` для работы с изображениями. Основные компоненты интерфейса: `QMainWindow`, `QLabel`, `QPushButton`, `QFileDialog`. Qt обеспечивает безопасную многопоточность через механизм сигналов и слотов.

Разработка выполнена в среде разработки Visual Studio Code [24]. Для сборки использовался CMake [25]. Для модульного тестирования применяется фреймворк Google Test [26].

3.2 Реализация проведения исследования

Время выполнения программы фиксируется с использованием библиотеки `std::chrono`, функции `high_resolution_clock::now()` [22] с точностью до

миллисекунд. Измеряется чистое время рендеринга кадра без учёта инициализации сцены и сохранения результата.

3.3 Реализация алгоритмов

Реализация функции алгоритма генерации части кадра представлена на листинге 1 в приложении Б.

Реализация функции получения луча из камеры представлена на листинге 2 в приложении Б.

Реализация функции получения цвета луча представлена на листинге 3 в приложении Б.

Реализация функции пересечения с приземлённым туманом на листинге 4 в приложении Б.

3.4 Пользовательский интерфейс

Главное окно программы представлено на рисунке 8. Оно разделено на две части: окно визуализации и окно настроек сцены и генерации.

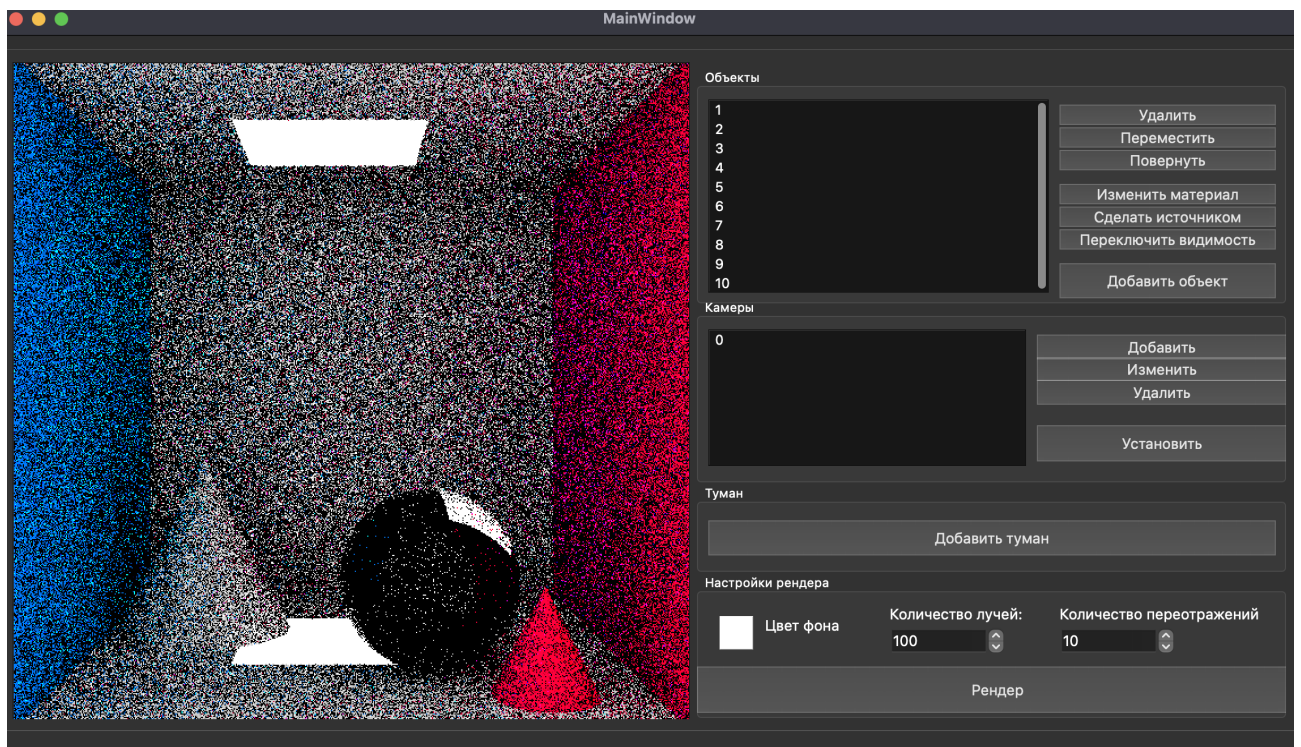


Рисунок 8 – главное окно программы

3.4.1 Панель настройки объектов

Панель настройки объектов позволяет, добавить, удалить, переместить, повернуть, изменить, сделать источником и переключить видимость объекта. Всплывающее окно добавления объектов представлено на рисунке 9. Пользователю необходимо указать геометрические параметры объекта и подтвердить выбор, после чего объект будет добавлен в сцену, и сразу же визуализирован.

Добавить объект

Тип объекта:
Сфера

4-уг. пирамида

Центр основания X: 0,00
Центр основания Y: 0,00
Центр основания Z: 0,00
Пол. ширины: 100,00
Высота: 100,00

3-уг. пирамида

V0 X: 0,00
V0 Y: 0,00
V0 Z: 0,00
V1 X: 0,00
V1 Y: 0,00
V1 Z: 100,00
V2 X: 100,00
V2 Y: 0,00
V2 Z: 0,00
Вершина X: 50,00
Вершина Y: 50,00
Вершина Z: 50,00

Четырёхугольник

Q X: 0,00
Q Y: 0,00
Q Z: 0,00
U X: -200,00
U Y: 0,00
U Z: 0,00
V X: 0,00
V Y: 0,00
V Z: 200,00

Треугольник

A X: 0,00
A Y: 0,00
A Z: 0,00
B X: 100,00
B Y: 0,00
B Z: 0,00
C X: 0,00
C Y: 100,00
C Z: 0,00

Коробка

Точка A X: 0,00
Точка A Y: 0,00
Точка A Z: 0,00
Точка B X: 0,00
Точка B Y: 0,00
Точка B Z: 0,00

Цилиндр

Ось X: 0,00
Ось Z: 0,00
Y0 (низ): 0,00
Y1 (верх): 200,00
Радиус: 100,00

Конус

Основание X: 0,00
Основание Y: 0,00
Основание Z: 0,00
Радиус: 100,00
Высота: 100,00

Сфера

Центр X: 0,00
Центр Y: 0,00
Центр Z: 0,00
Радиус: 100,00

Cancel OK

Рисунок 9 – окно добавление объекта

3.4.2 Панель изменения материала

Программа позволяет поменять материал объекта, на один из заранее установленных. Окно изменения материала продемонстрировано на рисунке 10.

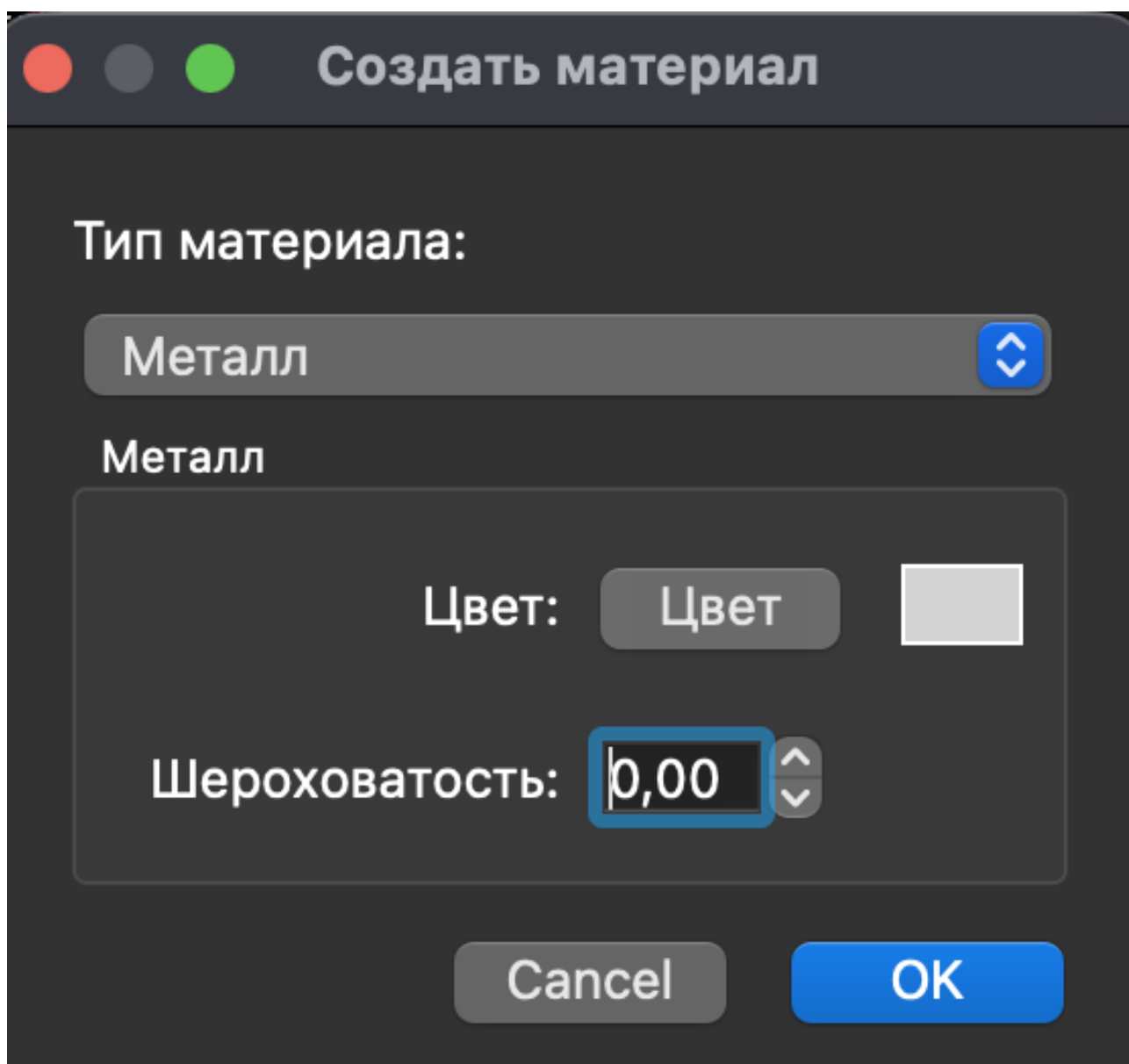


Рисунок 10 – окно изменения материала

3.4.3 Панель управления камерой

Панель управления камерой позволяет добавить, удалить, изменить положение и сделать камеру активной. При добавлении и изменении камеры необходимо указать позицию камеры, точку куда она смотрит, угол обзора, расстояние до точки фокуса и угол расфокусировки. Окно добавление камеры продемонстрировано на рисунке 11

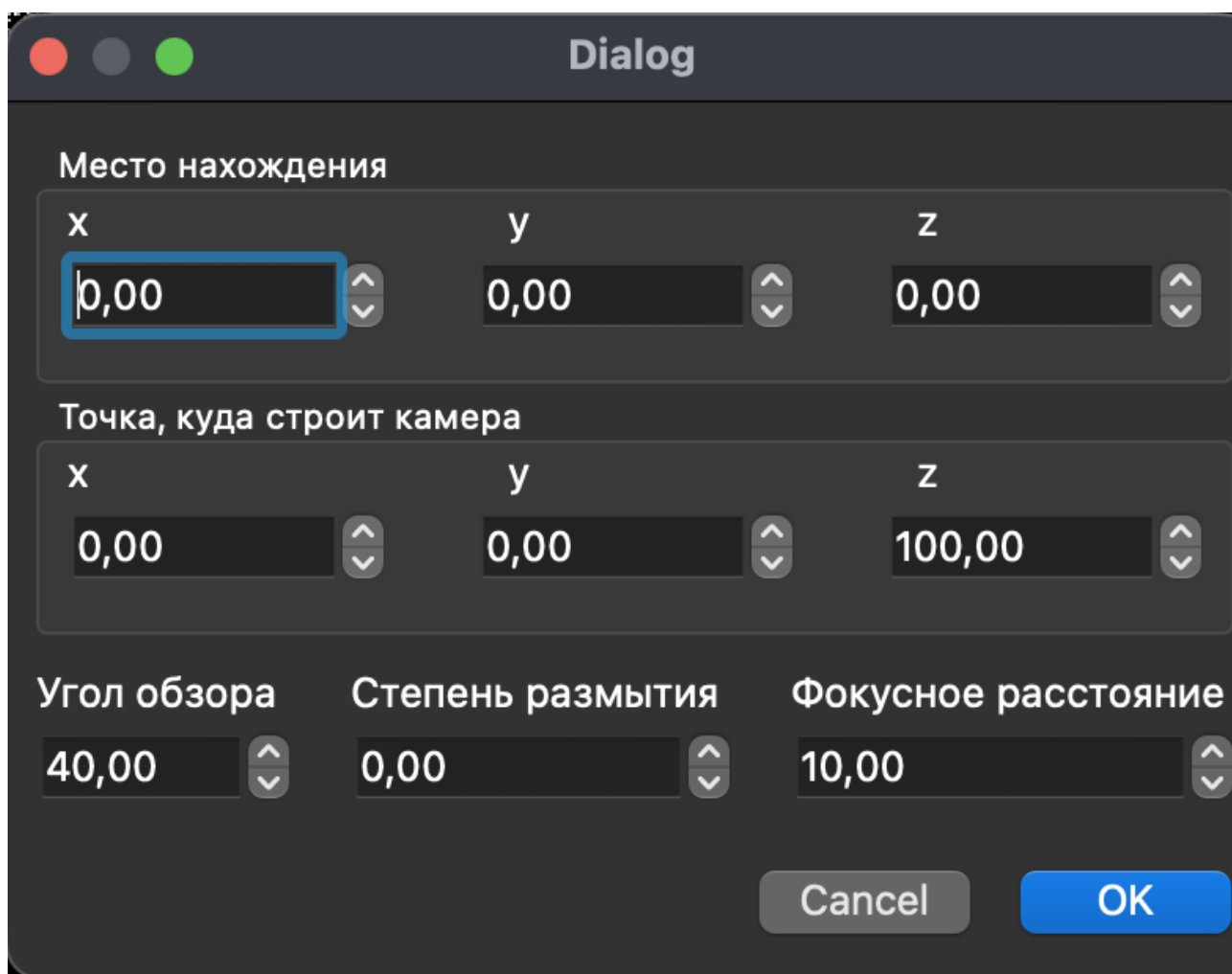


Рисунок 11 – окно добавление камеры

3.4.4 Панель настроек генерации

Панель настроек генерации изображения позволяет выбрать количество лучей на 1 пиксель, а также максимальную глубину рекурсии функции трассировки пути. При нажатии на кнопку начала генерации появляется всплывающее окно, в которое выводится результат. Процесс генерации изображения продемонстрирован на рисунке 12.

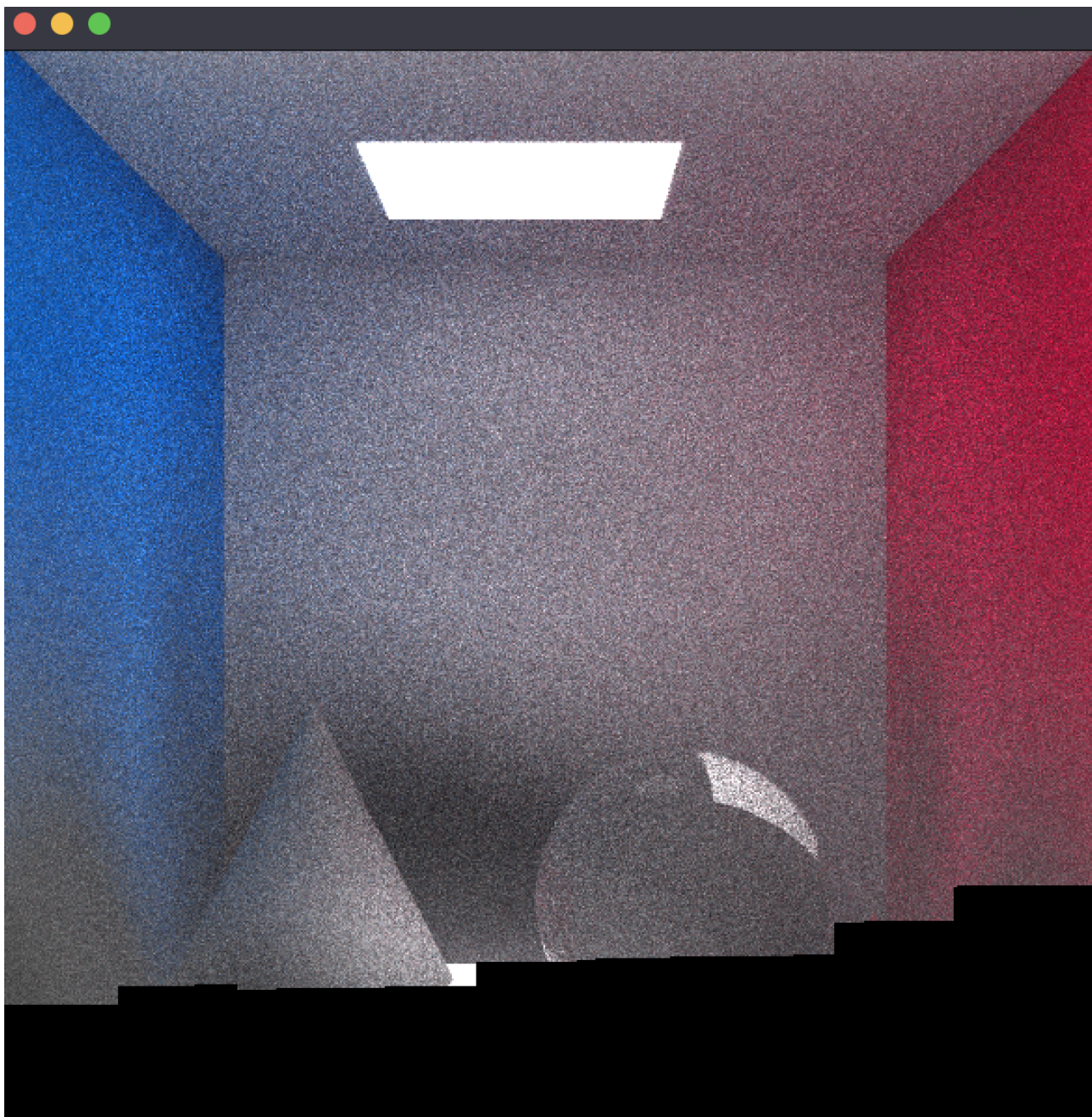


Рисунок 12 – процесс генерации изображения

Результат генерации изображение приведен на рисунке 13

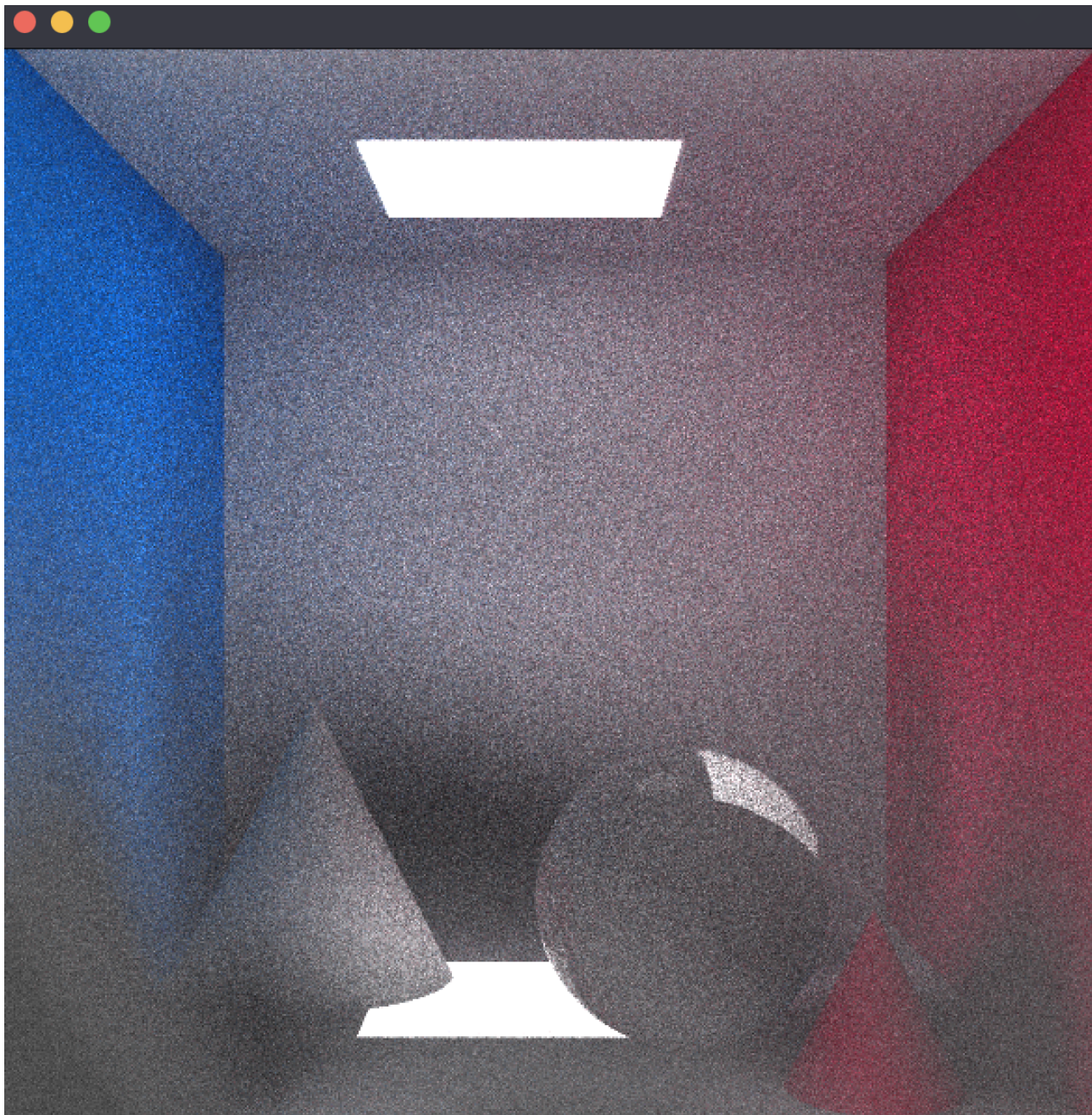


Рисунок 13 – результат генерации изображения

3.4.5 Панель добавления тумана

Панель позволяет добавить туман на сцену. Окно позволяет выбрать тип тумана, ограничивающий объем, плотность масштаб шума и фактор высоты. Окно добавления тумана приведено на рисунке 14

Fog Type: Наземный туман

Boundary Object ID: 10

Fog Color:

Parameters

Плотность: 0,900

Масштаб шума: 0,0000

Высотный фактор: 0,00000

OK Cancel

Рисунок 14 – Окно добавления тумана

Вывод

В данной части был обоснован выбор средств реализации, приведены исходные коды реализации используемых алгоритмов и приведены изображения интерфейса.

4 Исследовательская часть

4.1 Цель и методика исследования

Целью исследования является анализ влияния параметров алгоритма трассировки путей на производительность.

Исследованы две ключевые характеристики:

- количество лучей на пиксель при изменении от 10 до 100 с шагом 10;
- максимальная глубина рекурсии при изменении от 1 до 15 с шагом 2;

Для оценки производительности измеряется время рендеринга одного кадра фиксированного разрешения 400×400 пикселей. Каждое измерение повторяется 100 раз, результаты усредняются.

4.2 Технические характеристики

Исследование проводилось на компьютере со следующими характеристиками [27]:

- **Процессор:** Apple M3, 8 ядер (4 производительных + 4 энергоэффективных);
- **Оперативная память:** 16 ГБ LPDDR5;
- **Операционная система:** macOS Tahoe 26.1 (25B78);
- **Компилятор:** Apple Clang 16.0.0;

Во время замеров времени ноутбук был подключён к сети электропитания. Никакие пользовательские задачи в фоне не выполнялись. Оптимизация компилятора выключена флагом `-o0` [28].

4.3 Результаты исследований

4.3.1 Зависимость времени рендеринга от количества лучей

Результаты исследования зависимости времени выполнения реализации алгоритма трассировки пути для генерации одного кадра от количества испускаемых лучей на один пиксель представлены на рисунке 15. Глубина рекурсии была зафиксирована на десяти переотражениях.

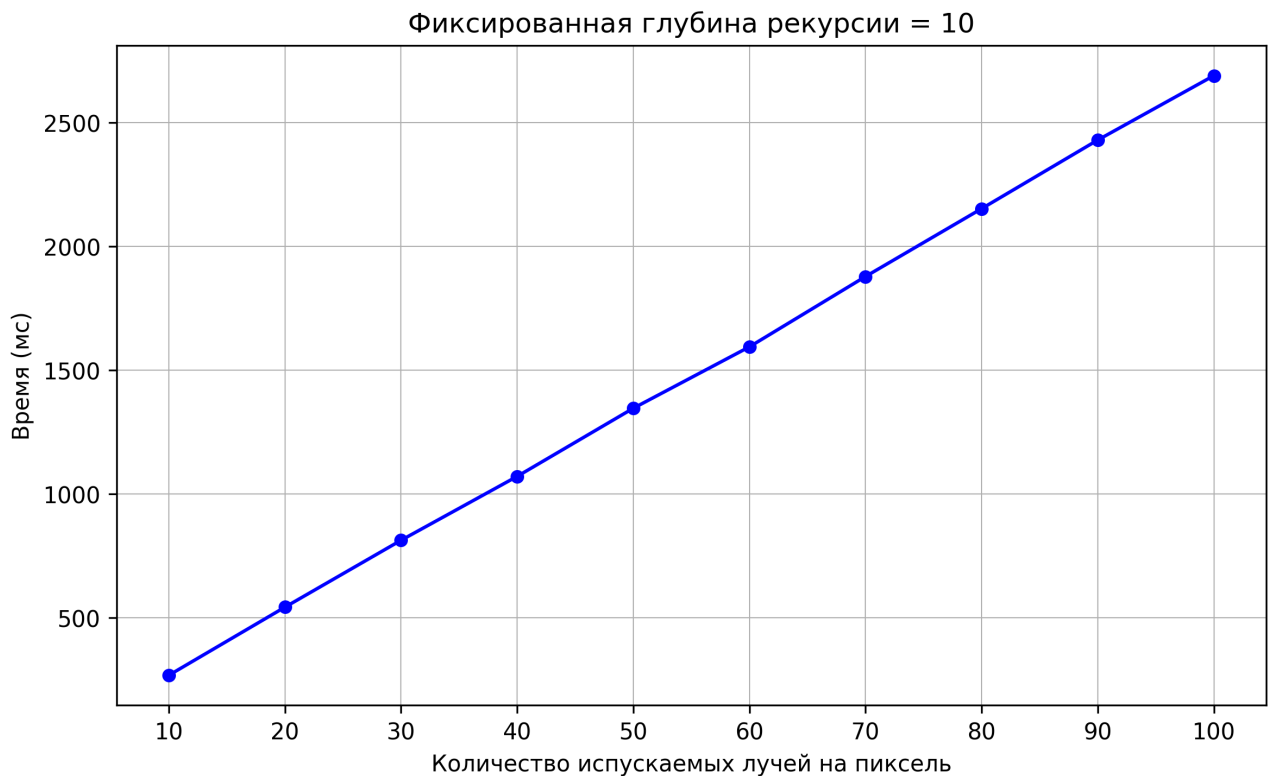


Рисунок 15 – график зависимости времени выполнения реализации алгоритма от количества лучей на пиксель

4.3.2 Зависимость времени генерации кадра от количества переотражений

Результаты исследования зависимости времени выполнения реализации алгоритма трассировки пути для генерации одного кадра от максимального количества переотражений одного луча представлены на рисунке 16. Количество испускаемых лучей на один пиксель было ограничено десятью единицами.

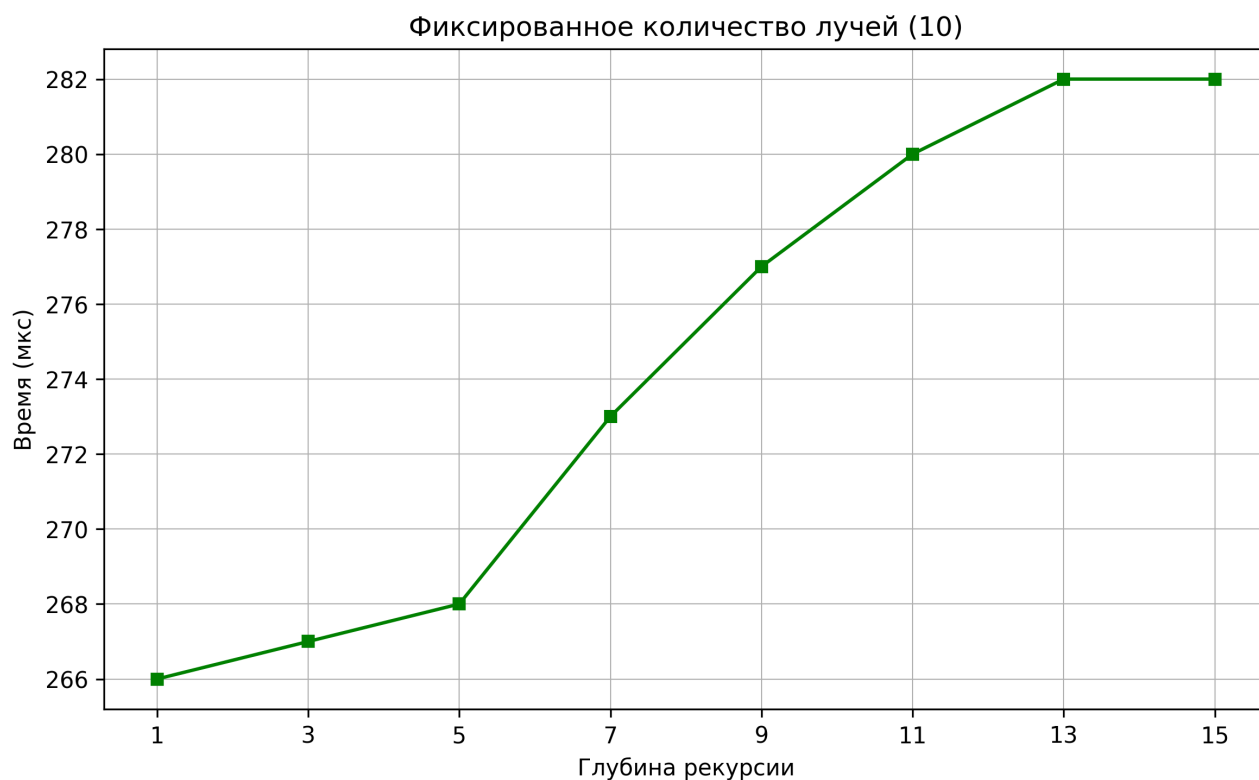


Рисунок 16 – график зависимости времени выполнения реализации алгоритма от максимального количества переотражений луча

4.4 Вывод

Было проведено исследование зависимости времени выполнения реализации алгоритма от его параметров, в ходе которого было выявлено:

- 1) время генерации одного кадра линейно зависит от количества лучей на пиксель;
- 2) время генерации одного кадра возрастает при увеличении глубины рекурсии: с увеличением числа допустимых переотражений наблюдается умеренный рост вычислительных затрат, однако данный эффект остаётся не критичным для общей производительности рендеринга.

ЗАКЛЮЧЕНИЕ

В результате курсовой работы разработано программное обеспечение для визуализации сцены из геометрических объектов и неравномерного тумана методом трассировки путей.

В ходе работы были решены следующие задачи:

- Проанализированы существующие методы моделирования тумана, проведён сравнительный анализ алгоритмов рендеринга. Для реализации выбраны экспоненциальная модель тумана и метод трассировки путей.
- Разработан алгоритм генерации изображения с поддержкой взаимодействия лучей с туманом, включающий стохастическое определение расстояния до взаимодействия, вычисление неоднородной плотности с использованием шума Перлина и экспоненциального спада по высоте.
- Реализован алгоритм генерации изображения методом трассировки путей.
- Реализована модель объёмного тумана с переменной плотностью на основе шума Перлина, создающая реалистичные турбулентные структуры и концентрацию у поверхности земли.
- Проведено исследование зависимости производительности от глубины рекурсии и количества лучей на пиксель. Установлена линейная зависимость от обоих параметров. Определены оптимальные параметры для интерактивной работы и финального рендеринга.

Разработанное программное обеспечение обеспечивает физически корректную визуализацию объёмного тумана с глобальным освещением и поддерживает редактирование параметров через графический интерфейс. Таким образом, все поставленные задачи решены, цель работы достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ayhan MUCUR Togan TONG. Effects of absorption and scattering values of liquids on global illumination // Megaron. 2025. Т. 20, № 2. С. 203–221.
2. Коэффициенты рэлеевского рассеяния в высоколегированных одномодовых германо- и фосфоро-силикатных световодах. / Лихачев М.Е., Семенов С.Л., Хопин В.Ф. [и др.] // Электронный журнал «ИССЛЕДОВАНО В РОССИИ». 2005.
3. Duggal S. Non-exponential Radiative Transfer for Volumetric Light Transport. https://shivamduggal4.github.io/images/pbr_rte.pdf. Дата обращения: 11.12.2025.
4. Investigation of LED Light Attenuation in Fog / K. Otas, V. J. Pakenas, A. Vaskys [и др.] // Electronics and Electrical Engineering. 2012. Т. 6, № 5. С. 47–52.
5. ФИКС И. И. ИСПОЛЬЗОВАНИЕ ГРАФИЧЕСКИХ ПРОЦЕССОРОВ ДЛЯ РЕШЕНИЯ ЗАДАЧИ РАСПРОСТРАНЕНИЯ СВЕТА В ДИФУЗНОЙ ФЛУОРЕСЦЕНТНОЙ ТОМОГРАФИИ МЕТОДОМ МОНТЕ-КАРЛО // Вестник Нижегородского университета им. Н.И. Лобачевского. 2011. № 4. С. 190–195.
6. Войтишек А. В. ОСНОВЫ МЕТОДА МОНТЕ-КАРЛО. <http://mmf.iis.nsk.su/sites/default/files/voytishekav-basics-of-mote-carlo-method.pdf>. Дата обращения: 11.12.2025.
7. Соколов Ю.А. Волкова Л.Л. МОДЕЛИРОВАНИЕ ТУМАНА В КОМПЬЮТЕРНОЙ ГРАФИКЕ // ЦИФРОВОЕ БУДУЩЕЕ ИННОВАЦИОННОЙ ЭКОНОМИКИ РОССИИ. 2016. Т. 8. С. 277–284.
8. А.В. Качурин, В.Ю. Ильичев. ОСНОВЫ ФОРМИРОВАНИЯ ВИЗУАЛЬНОЙ КАРТИНЫ ШУМА ПЕРЛИНА ПРИ ПРОГРАММИРОВАНИИ НА ЯЗЫКЕ PYTHON // E-Scio. 2021. Т. 10.
9. Fog Formulas.

- <https://learn.microsoft.com/ru-ru/windows/win32/direct3d9/fog-formulas>.
Дата обращения: 11.12.2025.
10. Miller G. Lecture 12: Probability Review.
<https://www.cs.cmu.edu/afs/cs/academic/class/15750-s19/ScribeNotes/lecture12.pdf>. Дата обращения: 16.12.2025.
 11. Peter Shirley Trevor David Black Steve Hollasch. Ray Tracing: The Next Week. 2025. April. – URL:
<https://raytracing.github.io/books/RayTracingTheNextWeek.html>.
 12. Задорожный А. Г. Модели освещения и алгоритмы затенения в компьютерной графике. НГТУ, 2020. Т. 80.
 13. Blinn J. F. Models of Light Reflection for Computer Synthesized Pictures // ACM SIGGRAPH Computer Graphics. Т. 11. 1977. С. 192–198.
 14. Picardello M. Mathematics for Global Illumination. 2012. – URL:
https://www.mat.uniroma2.it/~picard/SMC/didattica/materiali_did/Comp.Graph./Global_Illumination_x_Moscow.pdf.
 15. Corporation Microsoft. Растеризация.
<https://learn.microsoft.com/ru-ru/windows/win32/opengl/rasterizing>. Дата обращения: 11.12.2025.
 16. Quilez I. Rendering Worlds with Two Triangles with Raymarching.
<https://iquilezles.org/articles/>. 2008. Дата обращения: 11.12.2025.
 17. Обратная трассировка лучей. <http://ray-tracing.ru/articles164.html>.
Дата обращения: 11.12.2025.
 18. Hanrahan P. Monte Carlo Path Tracing.
<https://graphics.stanford.edu/courses/cs348b-01/course29.hanrahan.pdf>.
2005. Дата обращения: 11.12.2025.
 19. Volume Rendering for Developers: Summary of Equations.
https://math.arizona.edu/~tgk/mc/book_chap6.pdf. Дата обращения: 11.12.2025.

20. Wronski B. Volumetric Fog: Unified, compute shader based solution to atmospheric scattering. <https://bartwronski.com/publications/>. 2014. Дата обращения: 11.12.2025.
21. Pharr M., Jakob W., Humphreys G. Physically Based Rendering: From Theory to Implementation. 3rd изд. Morgan Kaufmann, 2016.
22. JTC1/SC22/WG21 ISO/IEC. ISO/IEC 14882:2020 Programming Languages — C++. <https://isocpp.org/files/papers/N4860.pdf>. 2020. Дата обращения: 12.12.2025.
23. Company The Qt. Qt 6 Documentation. <https://doc.qt.io/qt-6/>. 2024. Дата обращения: 12.12.2025.
24. Corporation Microsoft. Visual Studio Code. <https://code.visualstudio.com/>. Дата обращения: 12.12.2025.
25. Kitware Inc. CMake. <https://cmake.org/>. Дата обращения: 12.12.2025.
26. Inc. Google. GoogleTest — Google Testing and Mocking Framework. <https://github.com/google/googletest>. Дата обращения: 12.12.2025.
27. MacBook Air (13-inch, M3, 2024) - Tech Specs. <https://support.apple.com/en-us/118551>. Дата обращения: 12.12.2025.
28. Optimization level. <https://clang.llvm.org/docs/ClangCommandLineReference.html#optimization-level>. Дата обращения: 12.12.2025.

ПРИЛОЖЕНИЕ А

TODO Презентация к курсовой работе состоит из N слайдов. Структура файлов на накопителе: /src/ /report/ | --/report.pdf

ПРИЛОЖЕНИЕ Б

```
1  for (int i = tile.i; i < tile.end_i; i++)
2  {
3      if (cancel_running)
4          break;
5      for (int j = tile.j; j < tile.end_j; j++)
6      {
7          if (cancel_running)
8              break;
9          Color pixel_color(0, 0, 0);
10         for (int sample = 0; sample < _camera.samples_per_pixel;
11             sample++)
12         {
13             if (cancel_running)
14                 break;
15             Ray r = _camera.get_ray(j, i);
16             pixel_color += _camera.ray_color(r, _camera.max_depth,
17                 world, cancel_running);
18         }
19         color_matrix.at(i, j) = _camera.pixel_samples_scale *
20             pixel_color;
21     }
22 }
```

Листинг 1 – Реализация алгоритма рендера части карта

```
1 Ray get_ray(int i, int j) const
2 {
3     auto offset = sample_square();
4     auto pixel_sample = pixel00_loc + ((i + offset.x()) *
5         pixel_delta_u) + ((j + offset.y()) * pixel_delta_v);
6
7     auto ray_origin = (_camera.defocus_angle <= 0) ? center :
8         defocus_disk_sample();
9     auto ray_direction = pixel_sample - ray_origin;
10
11     return Ray(ray_origin, ray_direction);
12 }
```

Листинг 2 – Реализация алгоритма получения луча из камеры

```

1 Color ray_color(const Ray &r, int depth, const Hittable &world,
    volatile bool &cancel_running) const
2 {
3     if (cancel_running)
4         return Color(0, 0, 0);
5
6     if (depth <= 0)
7         return Color(0, 0, 0);
8
9     HitRecord rec;
10    if (!world.hit(r, Interval(0.001, infinity), rec))
11        return background;
12
13    Ray scattered;
14    Color attenuation;
15    Color color_from_emission = rec.mat->emitted(rec.u,
        rec.v, rec.p);
16    // Попытка рассеяния луча
17    if (!rec.mat->scatter(r, rec, attenuation, scattered))
18        return color_from_emission;
19
20    Color color_from_scatter = attenuation *
        ray_color(scattered, depth - 1, world, cancel_running);
21
22    return color_from_emission + color_from_scatter;
23 }

```

Листинг 3 – Реализация алгоритма получения цвета луча

```

1 Color ray_color(const Ray &r, int depth, const Hittable &world,
    volatile bool &cancel_running) const
2 bool hit(const Ray &r, Interval ray_t, HitRecord &rec) const
    override
3 {
4     HitRecord rec1, rec2;
5     if (!is_visible)
6         return false;
7     if (!boundary->hit(r, Interval::universe, rec1))
8         return false;
9     if (!boundary->hit(r, Interval(rec1.t + 0.0001, infinity),
        rec2))
10    return false;

```

```

11
12     auto t0 = std::fmax(rec1.t, ray_t.min);
13     auto t1 = std::fmin(rec2.t, ray_t.max);
14     if (t0 >= t1)
15         return false;
16
17     auto ray_len = r.direction().length();
18     auto dist = (t1 - t0) * ray_len;
19
20     auto hit_dist = -std::log(random_double()) / density;
21     if (hit_dist > dist)
22         return false;
23
24     auto t = t0 + hit_dist / ray_len;
25     Point3 p = r.at(t);
26
27     double raw_noise = noise.noise(scale * p);
28     double noise_val = 0.2 + 0.8 * std::fabs(raw_noise);
29     double y = p.y();
30     double height_factor = std::exp(-height_falloff * std::max(y,
31         -10.0));
32
33     double real_density = density * noise_val * height_factor;
34
35     if (real_density < 1e-5 || random_double() >
36         std::min(real_density, 1.0))
37         return false;
38
39     rec.t = t;
40     rec.p = p;
41     rec.normal = Vec3(0, 1, 0);
42     rec.front_face = true;
43     rec.mat = phase_function;
44     return true;
45 }

```

Листинг 4 – Реализация алгоритма нахождения пересечения с
приземленным туманом